高性能计算

HPC 第二关实验报告

stencil 与 GEMM 的性能瓶颈判断与优化决策

目　录

摘要

1. 实验环境与测试方法
    1.1 硬件与软件环境
    1.2 编译参数
    1.3 被测程序的命令行参数
    1.4 计时范围
    1.5 统计方法
    1.6 对照实验的控制变量
    1.7 数据稳定性与批次说明
2. 计算量与内存流量模型
    2.1 符号约定
    2.2 计算量
    2.3 内存流量模型
    2.4 各配置的模型计算值
    2.5 算术强度
    2.6 本节小结
3. stencil 实验数据与瓶颈分析
    3.1 数据质量检查
    3.2 三组尺寸的实测结果
    3.3 `row` 的耗时与格点数严格成正比
    3.4 核心分析：`column` 慢在哪里
    3.5 为什么 256×256 只有 4.09 倍
    3.6 横向对照与结论稳健性
    3.7 本节小结
4. GEMM 实验数据与瓶颈分析
    4.1 数据质量检查
    4.2 三组规模的实测结果
    4.3 两种实现的规模伸缩性
    4.4 加速比为何随规模增长
    4.5 核心分析：瓶颈已不在内存
    4.6 `tile` 扫描：调参空间已被算力上限封顶
    4.7 与 stencil 的横向对照
    4.8 本节小结
5. 算术强度与 Roofline 分析
    5.1 平台上下限的确定
    5.2 各实现的算术强度
    5.3 实测数据在平面上的位置
    5.4 三个需要显式声明的边界
    5.5 本节小结
6. 编译器证据与优化决策
    6.1 SIMD 证据之一：编译器向量化报告
    6.2 SIMD 证据之二：二进制中不存在 AVX 指令
    6.3 SIMD 证据之三：关断 AVX2 的消融实验
    6.4 三条证据的汇合
    6.5 为什么编译器没能向量化
    6.6 优化决策
    6.7 本节小结
7. 结论与局限性
    7.1 主要结论
    7.2 交付指标对照
    7.3 局限性
    7.4 后续工作

附录 A：原始数据表
    A.1 实验数据（共 22 个原始文件，均满足 `check=PASS`）
    A.2 成对汇总
    A.3 `tile` 扫描汇总（N = 512，`blocked`）
    A.4 数据溯源说明

附录 B：复现命令
    B.1 编译与实验命令
    B.2 第 6 节三份证据的采集命令
    B.3 工具可用性检查（可选）

项目地址

摘要

本报告以 stencil 与 GEMM 两类典型内核为对象，在 Intel i5-8250U 双核平台上固定编译参数与输入，配对测量 `row`/`column` 两种遍历、`naive`/`blocked` 两种 GEMM 实现以及 `tile` 扫描，共采集 22 组原始数据。通过建立计算量与内存流量模型、计算算术强度并进行 Roofline 定位，结合编译器向量化报告、目标文件反汇编和关断 AVX2 的消融实验，判断性能瓶颈的来源。结果表明：stencil 的 `column` 遍历在 1024^2 上比 `row` 慢 20.70 倍，两者的真实带宽处于同一量级，差距主要来自按列访问造成的约 16 倍读放大；GEMM 分块把模型流量降低约 42 倍，但 N=1024 时实测加速比仅 3.86 倍，N=512 时 `blocked` 的 GFLOP/s 为 3.06，已达到纯标量 FMA 上限的约 96%。三方面证据一致表明当前所有实现均未向量化，标量指令吞吐是共同天花板；`tile` 调参收益已饱和（约 1.6 倍）。据此给出优化优先级：stencil 先改遍历顺序，GEMM 再分块，最后通过内核改造实现向量化。

关键词：高性能计算；stencil；GEMM；缓存行；算术强度；Roofline 模型；SIMD；性能分析

1. 实验环境与测试方法

1.1 硬件与软件环境

项目	配置	来源
CPU	Intel(R) Core(TM) i5-8250U @ 1.60GHz	`/proc/cpuinfo`
可用核心数	2	`nproc`
L1 数据缓存	32 KB / 核	CPU 规格
L2 缓存	256 KB / 核	CPU 规格
L3 缓存	6144 KB（多核共享）	CPU 规格
缓存行大小	64 字节	CPU 规格
SIMD 支持	AVX2 + FMA（无 AVX-512）	`/proc/cpuinfo` 的 `flags` 字段
编译器	g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0	`g++ --version`
操作系统	Linux（Ubuntu 24.04）	系统信息

说明：缓存层级与缓存行大小未使用硬件计数器直接测量，取自 CPU 规格。第 3、4 节对实验现象的机理解释均以此为估算依据，相关结论已标注为估算而非实测。

1.2 编译参数

两个程序使用**完全相同**的编译命令：

```
g++ -std=c++17 -O2 -march=native -o stencil stencil.cpp
g++ -std=c++17 -O2 -march=native -o gemm    gemm.cpp
```

参数	作用
`-std=c++17`	任务指定的语言标准
`-O2`	开启优化。不加此参数两者均慢到失去对比意义，测得的将是"未优化的编译器"而非内存访问模式的差异
`-march=native`	允许编译器使用本机支持的 AVX2/FMA 指令

保持不变的原因：若两种实现使用不同优化等级，则无法区分性能差异来自"访问模式"还是"编译选项"。因此本报告所有对比实验中，两个程序的编译命令始终完全一致，仅改变被测程序的运行参数。

1.3 被测程序的命令行参数

程序	参数格式	本报告使用的取值范围

程序	参数格式	本报告使用的取值范围
`stencil`	`HEIGHT WIDTH STEPS row|column REPEATS`	边长 256 / 1024 / 2048；STEPS=20；REPEATS=5
`gemm`	`N naive|blocked TILE REPEATS`	N=256 / 512 / 1024；TILE=8…256；REPEATS=5

1.4 计时范围

吞吐数字只有在计时区间明确的前提下才有意义：若不清楚一段耗时包含了哪些工作，任何 GFLOP/s 或带宽数值都无法解释，也无法与其它实现比较。两个程序的计时区间如下。

下表为一**轮**（一次完整测量）内部的执行顺序。两程序均以 `for (run = -1; run < repeats; ++run)` 组织轮次，共执行 `repeats + 1` 轮：首轮为热身，耗时不计入统计。

表中“是否计入”列给出各阶段的计时边界。

阶段	stencil	gemm	是否计入 `compute_ms`
①	重置输入缓冲区：`a = input`，`b` 清零	`c` 清零	否
②	`Clock::now` 开始计时	`Clock::now` 开始计时	—
③	`for t in 0..steps-1`：`stencil_step` 与 `a.swap`	`gemm_naive` 或 `gemm_blocked`（blocked 含打包、清零、写回）	是
④	`Clock::now` 结束计时	`Clock::now` 结束计时	—
⑤	逐元素比对与边界零值检查，随后打印	逐元素比对，随后打印	否

上表即为"计时区包含与不包含"的表格化表述：**只有与 `Clock::now()` 成对的中间三个节点被计入 `compute_ms`**。

计时区**包含**与**不包含**的内容：

	stencil	gemm
计入	全部 `steps` 个时间步、缓冲区交换	所选实现的完整执行；blocked 实现包含打包（pack）、累加器清零与写回
不计入	输入重置、双精度参考计算、逐元素校验、边界检查、结果打印	`c` 清零、双精度参考计算、逐元素校验、结果打印

关键含义：FP64 参考计算与校验在计时区之外，但**仍属于进程开销**。故 `perf stat` 等整进程工具的统计结果**不可除以本报告的 `compute_ms`**——两者范围不一致。

1.5 统计方法

程序对每组参数执行 **`repeats` + 1 轮**：第 1 轮为热身（warmup），其耗时**不纳入统计**；其余 `repeats` 轮为正式样本。

程序输出六项统计量：`mean_ms`（算术平均）、`median_ms`（排序取中，偶数个取中间两者平均）、`stddev_ms`（**总体**标准差，除以 n 而非 n−1）、`min_ms` / `max_ms`（极值），以及样本数 `samples`。**所有吞吐指标（GFLOP/s、GUpdates/s、有效带宽）均以中位数为基准**，程序输出标注为 `throughput_time=median`。

1.6 对照实验的控制变量

固定量与变量安排如下表。

固定量	每次只改变的变量
输入数据 `seed=2026`；编译参数；问题规模与 `REPEATS`	`stencil`：`row` 与 `column`；`gemm`：`naive` 与 `blocked`；`gemm`：`TILE` 取值

- **输入完全相同**：两程序均使用固定种子 `seed=2026` 生成确定性输入，因此同规模下不同实现的输入逐位一致。
- **每轮均校验**：每一轮结束后都执行逐元素比对（`check=PASS/FAIL`），确保性能数据不以错误结果为代价。校验在计时区之外，不影响耗时。
- **一次只改一个变量**：比较 `row`/`column` 时固定规模、步数与编译参数；比较 `naive`/`blocked` 时固定 N 与编译参数；扫描 `tile` 时固定 N 与其余参数。

1.7 数据稳定性与批次说明

在数据采集过程中观测到同一配置、同一二进制的绝对耗时会随机器状态发生变化，因此需说明数据处理原则。

现象	观测结果
`stencil row`（1024×1024, steps=20）	中位数呈两簇：28.30 / 28.71 ms 与 34.31 / 34.81 ms，簇内偏差约 1.5%，簇间相差约 21%
`stencil column`（同配置）	分布于 585.80 ~ 668.35 ms
`row`/`column` 比值	四次独立配对测量分别为 20.70×、21.63×、21.95×、19.20×，均值 20.87×，区间 19.20 ~ 21.95×

处理原则：

1. **主要结论基于同批次配对数据**。第 3 节采用同一批次内连续测得的 `row` 与 `column`（28.30 ms / 585.80 ms，比值 20.70×）作为主结论；该批次采样时间为 10-12 22:53，两组的 5 个样本各自独立且波动很小。
2. **其余批次仅用于稳定性验证**。四次配对测量的比值落在 19.20 ~ 21.95× 区间，均值 20.87×（见附录 A），表明结论对测量批次与机器状态不敏感。其中第三次配对的 `row` 与 `column` 分属相邻批次，故仅用于佐证趋势，不作为主结论。
3. **不跨批次比较绝对耗时**。

推测性解释（未经硬件计数器验证）：`row` 受 CPU 频率影响显著（顺序访问下延迟被掩盖，瓶颈偏向计算），`column` 受内存限制、对频率不敏感。

---

2. 计算量与内存流量模型

在解释任何吞吐数字之前，必须先确定三件事：**每个实现做了多少次浮点运算、理想情况下要搬运多少字节、以及这些数字依赖什么假设**。本节建立后续所有分析所用的两把"尺子"。

2.1 符号约定

符号	含义	stencil	gemm
`h`, `w`	网格高度、宽度	命令行参数	—
`steps`	时间步数	命令行参数	—
`n`	方阵边长	—	命令行参数
`tile`	分块边长	—	命令行参数
`updates`	格点更新总次数	`(h-2)×(w-2)×steps`	—

2.2 计算量

stencil

内部格点数为 $(h-2)\times(w-2)$（边界一圈固定为零，不参与计算），每步更新一次，故：

```
updates = (h-2) × (w-2) × steps
FLOP    = 6 × updates
```

每格点 6 FLOP 的来源（`stencil.cpp` 第 14 行）：四个邻居两两相加（3 次加法）+ 乘 `0.125f`（1 次乘法）+ 乘 `0.5f` 并相加（1 乘 + 1 加）。

gemm

输出矩阵有 n^2 个元素，每个元素需沿 k 维度做 n 次"乘加"，每次乘加计 2 FLOP：

```
FLOP = 2 × n³
```

该计数只包含乘加运算，不含打包、索引与循环控制开销。**两种实现的计算量完全相同**，因此性能差异不可能来自计算量。

2.3 内存流量模型

模型 A：理想空间复用模型

程序内置的模型（`stencil.cpp` 第 95 行）假设：**每个输入元素在每一步中只从内存读取一次，每个输出元素只写入一次。**

实现	理想流量公式	每更新一格的字节数
stencil	$8 \times updates$	8 字节（4 读 + 4 写）
gemm naive	$8n^3$	随 n^3 增长
gemm blocked	$12n^3/tile + 3n^2$	随 n^3/tile 增长

该模型的隐含假设有两条，缺一不可：

1. **空间局部性成立**：一次取回的缓存行（64 字节 = 16 个 float）能被充分使用；
2. **复用数据驻留缓存**：需要重复使用的数据在再次被访问前未被逐出。

blocked 流量公式推导。 `gemm_blocked` 对每个输出块 `(ii, jj)` 沿 k 方向遍历全部 `n/tile` 个深度块，三个矩阵搬运量不同：

矩阵	content	被搬运的次数	字节数
A	行块 `(ii, :)` 在每个列块下都要重读	`n/tile` 次	$4n^2 \cdot n/tile$
B	列块 `(:, jj)` 在每个行块下都要重读	`n/tile` 次	$4n^2 \cdot n/tile$
C	每个输出元素写回一次	1 次	$4n^2$

合计：

```
blocked 字节数 = 4n³/tile + 4n³/tile + 4n² = 12n³/tile + 3n²
```

关键点：A、B 各被重复搬运 `n/tile` 次，均为 n^3 量级；C 项仅 n^2 量级。遗漏前两项会严重低估 blocked 的流量。

模型 B：访问模式修正模型

模型 A 的假设是否成立，**取决于最内层循环的地址步长**。设最内层每前进一次，地址前进 s 个元素：

最内层步长	缓存行利用率	读流量放大倍数

最内层步长	缓存行利用率	读流量放大倍数
`s = 1`（连续）	16/16 = 100%	×1
`s ⩾ 16`（跨行）	1/16 = 6.25%	**×16**

由此得到两个程序各实现的每格点字节数：

实现	最内层步长	读（字节/点）	写（字节/点）	合计	相对模型 A
stencil `row`	1	4	4	**8**	×1.0
stencil `column`	`w`	4 × 16 = **64**	4	**68**	**×8.5**

放大倍数 16 的推导：一次读取请求只取回一个 float（4 字节），但内存子系统的最小传输单位是一条缓存行（64 字节）。当 `w ⩾ 16` 时，步长为 `w` 的相邻访问落在不同缓存行中，每条缓存行仅被使用 4 字节：

```
64 字节 ÷ 4 字节 = 16
```

因此按列遍历的**读流量放大约 16 倍**。写操作本身是顺序的，不受此惩罚，故**总流量放大约 (64+4)/(4+4) ≈ 8.5 倍**。区分"读放大"与"总放大"是必要的：把 16 倍直接套到总流量上会高估其影响。

模型 B 的适用边界

模型 B 假设"被浪费的 15 个元素一定不会被再次使用"。该假设仅在**工作集超出缓存容量**时成立：

网格尺寸	单步工作集（3 个数组）	与 6 MB L3 的关系	模型 B 是否适用
256×256	约 0.75 MB	可容纳	❌ 可能高估
1024×1024	约 12 MB	超出	✅ 适用
2048×2048	约 48 MB	远超	✅ 适用

该边界条件将在第 3 节用实验数据检验。

2.4 各配置的模型计算值

以下数值由上述公式直接算出（**非硬件计数器实测**），构成第 3、4 节全部理论预测与对比比例的基准。

stencil（steps = 20）

尺寸	updates	FLOP	模型 A 字节数	column 的模型 B 字节数
256×256	1.29×10^6	7.75×10^6	10.3 MB	88 MB
1024×1024	2.09×10^7	1.25×10^8	167 MB	1.42 GB
2048×2048	8.36×10^7	5.01×10^8	669 MB	5.68 GB

两种遍历的**每格点**字节数差异，图形化表示如下：

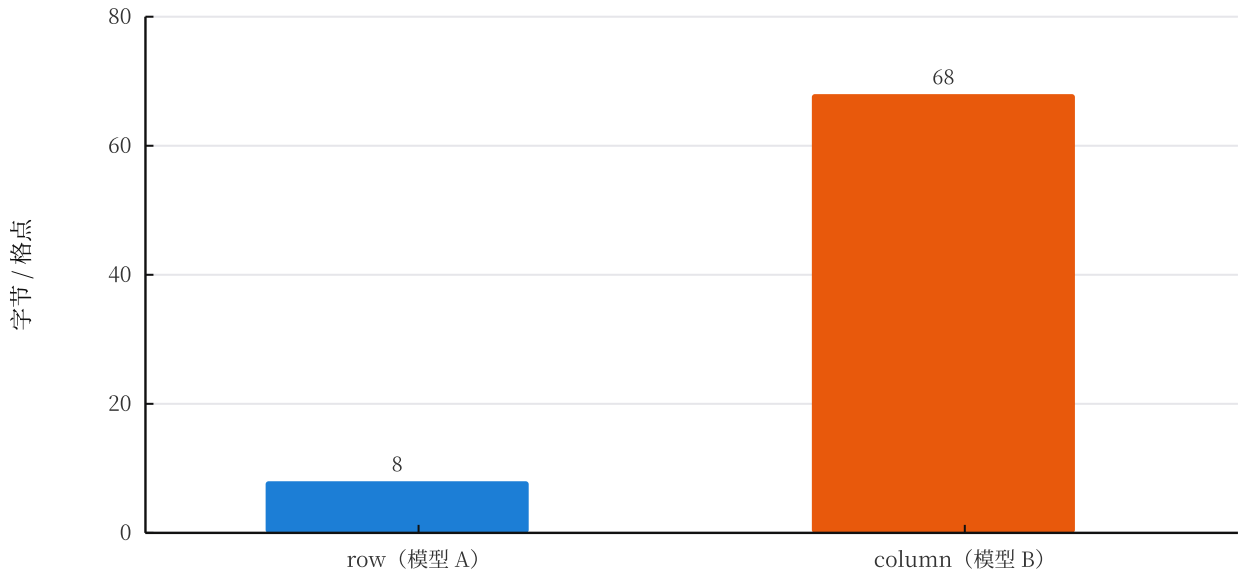


图 1 stencil 两种遍历的模型预测每格点搬运量

`row` 与 `column` 的差值为 `68 - 8 = 60` 字节/格点，即 `68 ÷ 8 = 8.5` 倍。注意图中只体现**总流量**的放大；若只看读操作，放大倍数为 `64 ÷ 4 = 16`（见 2.3 节）。

gemm

N	tile	FLOP	naive 字节数（$8n^3$）	blocked 字节数	模型预测流量比
256	64	3.36×10^7	134 MB	3.3 MB	40.2×
512	8	2.68×10^8	1.07 GB	202.1 MB	5.3×
512	32	2.68×10^8	1.07 GB	51.1 MB	21.0×
512	64	2.68×10^8	1.07 GB	26.0 MB	41.4×
512	256	2.68×10^8	1.07 GB	7.1 MB	151.7×
1024	64	2.15×10^9	8.59 GB	204.5 MB	42.0×

`n = 512` 时流量比随 `tile` 的变化，图形化表示如下：

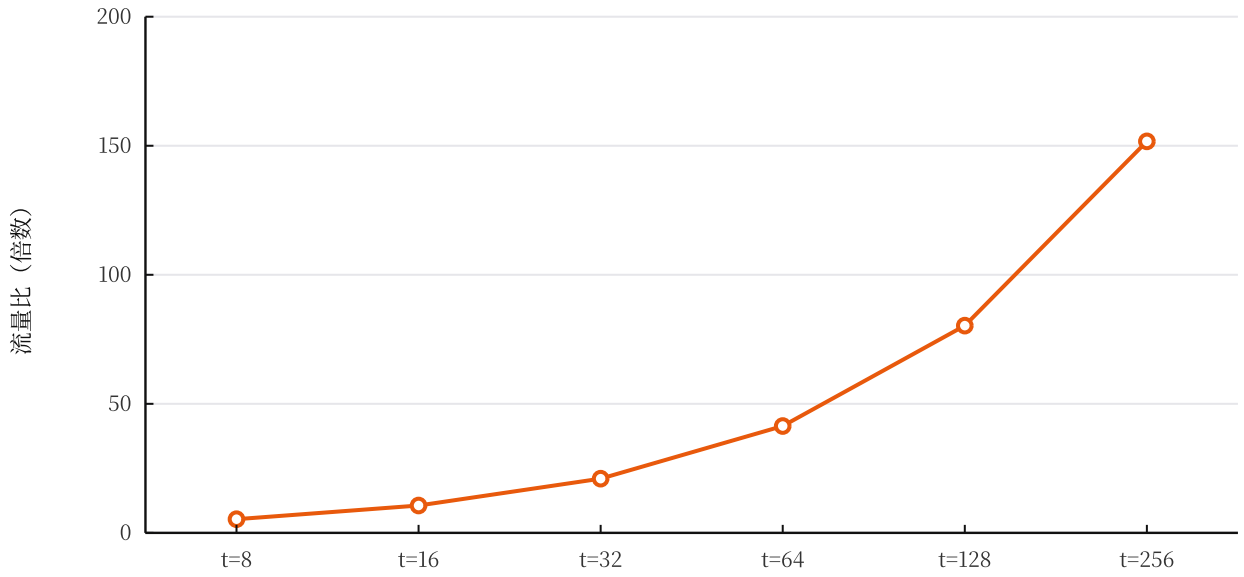


图 2 gemm blocked 相对 naive 的模型流量比随 tile 的变化（n = 512）

图中 `t = 8` 的柱子较矮，是**线性轴的如实呈现**：该点流量比仅 5.3 倍，相对于 `t = 256` 的 151.7 倍确实很小。这正说明分块的收益高度依赖 `tile` 的取值，而非"分块即有效"。

blocked 的模型流量与 `tile` **成反比**（$12n^3/tile$ 主导），故增大 `tile` 的边际收益递减；`tile = 8` 时仅比 naive 少搬 5.3 倍，说明分块并非无条件有效。

此处必须强调一个容易误读之处：模型预测的"流量比"**不等于**模型预测的"加速比"。加速比取决于内存受限部分占总耗时的比例；若实现同时受限于计算吞吐，则流量下降无法等比转化为时间缩短。第 4 节将用实测数据说明这一点。

2.5 算术强度

算术强度定义为每搬运一字节所执行的浮点运算次数，是判断实现受限于计算能力还是内存带宽的核心指标：

```
算术强度 I = FLOP ÷ 搬运字节数   [FLOP/Byte]
```

实现	分子	分母	算术强度
stencil（模型 A）	$6 \times updates$	$8 \times updates$	**0.75**（与规模无关）
gemm naive	$2n^3$	$8n^3$	**0.25**（与规模无关）
gemm blocked	$2n^3$	$12n^3/tile + 3n^2$	**10.34**（n=512, tile=64）；当 $n/tile$ 较大时 $\approx tile/6$

stencil 的算术强度恒为 0.75，与尺寸、步数**无关**（`updates` 被约去），结构上即偏向访存密集。gemm blocked 则可通过增大 `tile` 提高算术强度；**该值同时依赖 `n`**（分母的 $3n^2$ 项在小 `n` 下不可忽略），详见 5.2 节。上述强度值与机器脊点的比较见第 5 节。

2.6 本节小结

1. **计算量**：stencil 为 $6(h-2)(w-2)steps$ FLOP；gemm 两种实现均为 $2n^3$ FLOP。计算量不构成实现间的差异来源。
2. **流量模型分两层**：模型 A（理想复用）给出理论下限；模型 B 按最内层步长修正，揭示按列遍历的读流量放大 16 倍、总流量放大 8.5 倍。对 gemm，blocked 的流量为 $12n^3/tile + 3n^2$，与 `tile` 成反

比。

3. **两个模型都是带假设的预测，不是实测值**。模型 A 对 `column` 及大 `n` 的 naive 实现失效，模型 B 在小工作集下高估。第 3、4 节的任务即用实测数据检验其适用边界。
4. **算术强度**：stencil 恒为 0.75，gemm naive 恒为 0.25；gemm blocked 随 `tile` 与 `n` 变化（n=512 时 `tile = 8/64/256` 分别为 1.33 / 10.34 / 37.93）。这是第 5 节瓶颈判断的量化依据。
5. **模型预测的是流量比，不是加速比**。二者只有在实现完全受限于内存时才相等；第 4 节将显示 gemm 的实际加速远低于流量比，这本身就是一条关于真实瓶颈的线索。

---

3. stencil 实验数据与瓶颈分析

3.1 数据质量检查

在比较性能之前，先确认两种遍历算出的结果一致——否则性能差异没有意义。

尺寸	`row` checksum	`column` checksum	是否一致	`max_abs_error`
256×256	31647.2	31647.2	✅	1.20×10^{-7}
1024×1024	519158	519158	✅	1.44×10^{-7}
2048×2048	2.08718×10^{6}	2.08718×10^{6}	✅	1.51×10^{-7}

三组配置的 `checksum` 在两种遍历下**逐位相同**。这说明：

> `row` 与 `column` 求解的是同一个数学问题，输入、步数、编译参数完全相同，**唯一差异是网格的遍历次序**。因此后续观测到的所有性能差异，均不可能来自计算量或结果正确性。

所有运行的 `check` 均为 `PASS`，`max_abs_error` 在 10^{-7} 量级，符合 FP32 相对 FP64 参考的预期精度。

3.2 三组尺寸的实测结果

固定 `steps = 20`、`repeats = 5`、编译参数一致，仅改变网格尺寸与遍历次序。计时区包含全部 20 个时间步与缓冲区交换，不包含输入重置、FP64 参考计算与校验。

尺寸	`row` 中位数	`column` 中位数	比值	`row` 标准差占比	`column` 标准差占比
256×256	**1.704 ms**	**6.973 ms**	**4.09×**	9.17%	3.35%
1024×1024	**28.298 ms**	**585.796 ms**	**20.70×**	0.30%	0.54%
2048×2048	**114.001 ms**	**2354.190 ms**	**20.65×**	1.68%	0.87%

三点直接读出的结论：

1. **`row` 始终更快**，与理论预期一致，方向无误。
2. 在 1024 及以上尺寸，`column` 稳定地慢 **约 20.7 倍**，且该比值在 1024 与 2048 之间几乎不变（20.70× 与 20.65×）。
3. **256×256 的比值只有 4.09×**，显著低于大尺寸——这不是噪声，而是工作集尺寸导致的系统性差异，将在 3.5 节解释。

两种遍历的中位数耗时随网格尺寸的变化如下（纵轴为线性刻度）：

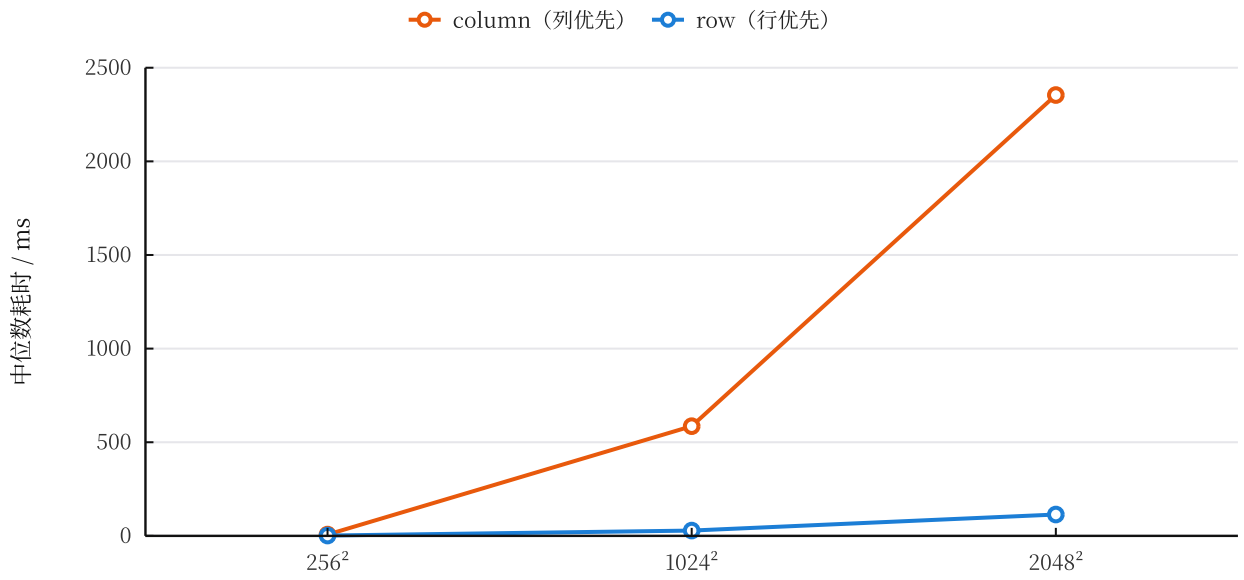


图 3 stencil 两种遍历的中位数耗时随网格尺寸的变化

图 3 中上方曲线为 `column`、下方曲线为 `row`。**纵轴为线性刻度时 `row` 几乎贴住横轴**——这一视觉落差本身就是两种实现差距的直观体现（1024^2 处相差 20.70 倍）。`column` 的曲线明显上凸，说明其耗时增长快于格点数。

3.3 row 的耗时与格点数严格成正比

以 1024 与 2048 两组为例（格点数之比为 4:1）：

对比	格点数之比	耗时之比	偏差
`row`：1024 → 2048	4.00	114.001 ÷ 28.298 = **4.03**	+0.7%
`column`：1024 → 2048	4.00	2354.190 ÷ 585.796 = **4.02**	+0.5%

两种遍历的耗时都**几乎精确正比于格点总数**，说明每格点成本恒定，均无随规模恶化的额外开销。`row` 的 GFLOP/s 在三种尺寸上高度一致：

尺寸	row GFLOP/s	column GFLOP/s
256×256	4.54	1.11
1024×1024	4.43	0.21
2048×2048	4.41	0.21

`row` 稳定在 4.4 ~ 4.5 GFLOP/s，`column` 稳定在 0.21 GFLOP/s（256 除外）。**性能与规模无关**这一特征，说明瓶颈来源是逐格点的固定成本，而非缓存容量随规模变化。

3.4 核心分析：column 慢在哪里

程序的"有效带宽"指标对 column 失效

程序输出的 `effective_bandwidth_GB/s` 按 `8 × updates ÷ median_ms` 计算（`stencil.cpp` 第 95 行），其分子基于"理想空间复用"假设：每个输入元素每步只从内存读取一次、每个输出只写入一次，即每格点 8 字节。

按此公式计算三组结果：

尺寸	row 有效带宽	column 有效带宽
256×256	6.06 GB/s	1.48 GB/s
1024×1024	5.91 GB/s	**0.285 GB/s**
2048×2048	5.88 GB/s	**0.285 GB/s**

`column` 的 0.285 GB/s 在物理上不可能成立：该数值约为 285 MB/s，低于现代机械硬盘的顺序读写速度，而本机内存的可用带宽在 5 GB/s 量级以上（`row` 的实测值即为佐证）。因此这个数字**不能解释为"带宽不足"**。

原因在于该公式的分子与分母来自两个不同的假设层面：

```
有效带宽 = 8 × updates  ÷  实测耗时
           └──────┬──────┘     └──┬──┘
          模型假设值(固定)    真实测量值
```

分子 `8 × updates` 对 `row` 与 `column` **完全相同**——它是编译期常量，与被测实现的访问模式无关。因此该指标在假设成立时才是"带宽"，在假设失效时只是两个不同量纲数值的商。

用修正模型还原真实带宽

按第 2 节模型 B，`column` 的内层步长为 `w`，每条 64 字节缓存行仅用 4 字节，故读放大 16 倍、每格点 68 字节（`4×16 + 4`），总流量约 `68 × 2.09 × 10⁷ ≈ 1.42 GB`，与 `row` 的 8 字节/点相差 8.5 倍。据此还原真实带宽：

实现	实际搬运量	实测耗时	真实带宽	程序报出的"有效带宽"
`row`	167 MB	28.298 ms	**5.91 GB/s**	5.91 GB/s
`column`	1420 MB	585.796 ms	**2.42 GB/s**	**0.285 GB/s** ❌

两个实现的真实带宽处于同一量级（2.42 与 5.91 GB/s，相差约 2.4 倍），而程序报出的数值相差 20.7 倍。

结论

> `column` 慢 20.7 倍的主要原因**不是它"用不上带宽"，而是它实际搬运的字节数多得多**：由于内层步长为 `w`，缓存行利用率降至 4/64 = 6.25%，读流量放大约 16 倍，计入不放大写的写流量后总流量放大约 8.5 倍。程序的 `effective_bandwidth_GB/s` 对 `column` **不具物理意义**，其数值低于硬件带宽的物理下限，不能作为"带宽不足"的证据。程序自身也在输出末行标注了该指标的性质：`bandwidth_model=ideal_spatial_reuse bytes_per_update=8`。

模型与实测的残差

模型预测总流量放大 8.5 倍，实测耗时放大 20.70 倍，两者相差约 **2.4 倍**。该残差有明确来源，属于模型未覆盖的部分：

1. **流量比不等于时间比**。只有当实现完全受限于带宽时，两者才相等。若 `column` 同时受限于访存延迟，其时间代价会高于流量代价。
2. **延迟因素（推测）**。`column` 的内层步长为 `w`（1024 个元素 = 4096 字节），硬件预取器难以识别此类非连续步长，可能无法有效隐藏内存延迟；而 `row` 的步长为 4 字节，预取器容易识别。本报告未使用硬件计数器验证此点，故仅作为推测提出。

本报告不将 20.70 倍全部归因于流量放大，而是明确区分：**流量放大 8.5 倍有模型支撑，剩余约 2.4 倍归因于延迟效应，属未经硬件计数器验证的推测。**

以上解释了 `column` 慢的原因，但尚未回答一个更进一步的问题：**两个实现各自属于何种瓶颈**。这需要借助算术强度与脊点作统一判定，见第 5 节。

3.5 为什么 256×256 只有 4.09 倍

这是三组数据中最值得解释的一组。按 3.4 节的模型，总流量放大 8.5 倍；若 `column` 完全受限于流量，其耗时应为 `1.704 × 8.5 ≈ 14.5 ms`，而实测只有 **6.973 ms**。

反向验证：实测 6.973 ms 对应搬运量 `6.973 ms × 5.91 GB/s ≈ 41 MB`，即每格点约 **32 字节**，显著低于模型 B 预测的 68 字节。

原因是工作集可以驻留缓存。

尺寸	单步工作集（3 个数组）	与 6 MB L3 的关系	模型 B 预测	实测现象
256×256	约 0.75 MB	可容纳	8.5×	**4.09**×（低于预测）
1024×1024	约 12 MB	超出	8.5×	**20.70**×（高于预测）
2048×2048	约 48 MB	远超	8.5×	**20.65**×（高于预测）

工作集小于缓存容量时，被"浪费"的 15 个元素仍驻留缓存并可被复用，故惩罚未完全兑现。**这验证了 2.3 节对模型 B 适用边界的预判**：该假设在 1024 及以上成立，在 256 尺寸下失效。

3.6 横向对照与结论稳健性

与 naive GEMM 的横向对照

`gemm` 的 naive 实现在结构上与 stencil 的 `column` 同源：内层步长均为跨行（分别为 `n` 与 `w`），都会造成缓存行浪费。第 4 节将给出其量化结果，可与此处对照。

结论对机器状态不敏感

`stencil row` 的中位数在不同批次间出现过两簇取值（28.30 / 28.71 ms 与 34.31 / 34.81 ms），簇间相差约 21%，推测与移动平台 CPU 热降频有关（详见 1.7 节）。但 `row`/`column` 的**比值**在四次独立配对测量中稳定在 19.20 ~ 21.95×（均值 20.87×），说明核心结论不依赖具体批次。

此外，`row` 与 `column` 的稳定性本身也有差异：

	四次测量跨度	波动幅度
`row`	28.30 ~ 34.81 ms	约 21%
`column`	585.80 ~ 668.35 ms	约 12%

推测：`row` 的性能受 CPU 频率影响更明显（顺序访问下内存延迟被有效掩盖，瓶颈偏向计算），而 `column` 受内存子系统限制，对频率变化不敏感。这意味着访存模式不仅影响平均性能，也影响性能的可预测性。（此项为推测，未经硬件计数器验证。）

3.7 本节小结

1. **正确性**：三组尺寸下两种遍历的 `checksum` 逐位一致，性能差异与计算量无关。
2. **性能**：`row` 始终更快；1024 与 2048 尺寸下 `column` 稳定慢约 **20.7 倍**。
3. **归因**：`column` 的内层步长为 `w`，缓存行利用率仅 6.25%，读流量放大 16 倍、总流量放大 8.5 倍。模型与实测残差约 2.4 倍，归因于访存延迟，属推测。
4. **指标辨析**：程序输出的 `effective_bandwidth_GB/s` 对 `column` 失效（0.285 GB/s 低于物理下限）；用修正模型还原后，两者的**真实带宽分别为 5.91 与 2.42 GB/s，处于同一量级**。
5. **边界条件**：模型 B 仅在超出缓存容量的大尺寸下成立；256×256 因工作集可驻留 L3，实测比值仅 4.09 倍，低于模型预测。

6. **稳健性**：核心倍数在四次独立测量中稳定于 19.20 ~ 21.95×，不受机器状态影响。

---

4. GEMM 实验数据与瓶颈分析

4.1 数据质量检查

N	naive checksum	blocked checksum	是否一致	max_abs_error
256	4.19794×10^{6}	4.19794×10^{6}	✅	5.93×10^{-5}
512	3.3519×10^{7}	3.3519×10^{7}	✅	1.92×10^{-4}
1024	2.68355×10^{8}	2.68355×10^{8}	✅	5.57×10^{-4}

三组配置下两种实现的 `checksum` 逐位一致。注意 `naive`（`i-j-k`）与 `blocked`（内层 `i-k-j`）的**归约顺序并不相同**，在此前提下 checksum 仍完全一致，说明二者求解同一乘法，**性能差异与计算量、正确性无关**。`max_abs_error` 随归约长度从 5.93×10^{-5} 增至 5.57×10^{-4}，符合 FP32 累加预期。

4.2 三组规模的实测结果

固定 `tile = 64`、`repeats = 5`、编译参数一致，仅改变规模与实现。

N	naive 中位数	blocked 中位数	加速比	naive GFLOP/s	blocked GFLOP/s
256	**19.826 ms**	**11.632 ms**	**1.70×**	1.69	2.88
512	**265.565 ms**	**87.712 ms**	**3.03×**	1.01	3.06
1024	**2776.290 ms**	**719.567 ms**	**3.86×**	0.77	2.98

两点直接读出的结论：

1. `blocked` 在三种规模下**始终更快**，方向与理论预期一致。
2. **加速比随规模单调增长**（1.70× → 3.03× → 3.86×），而非恒定——这需要解释，见 4.4 节。

`blocked` 的 GFLOP/s 在三种规模下高度稳定（2.88 / 3.06 / 2.98），而 `naive` 随规模**下降**（1.69 / 1.01 / 0.77）。这一升一降的反差是本节分析的主线。

两种实现的中位数耗时随规模的变化如下：

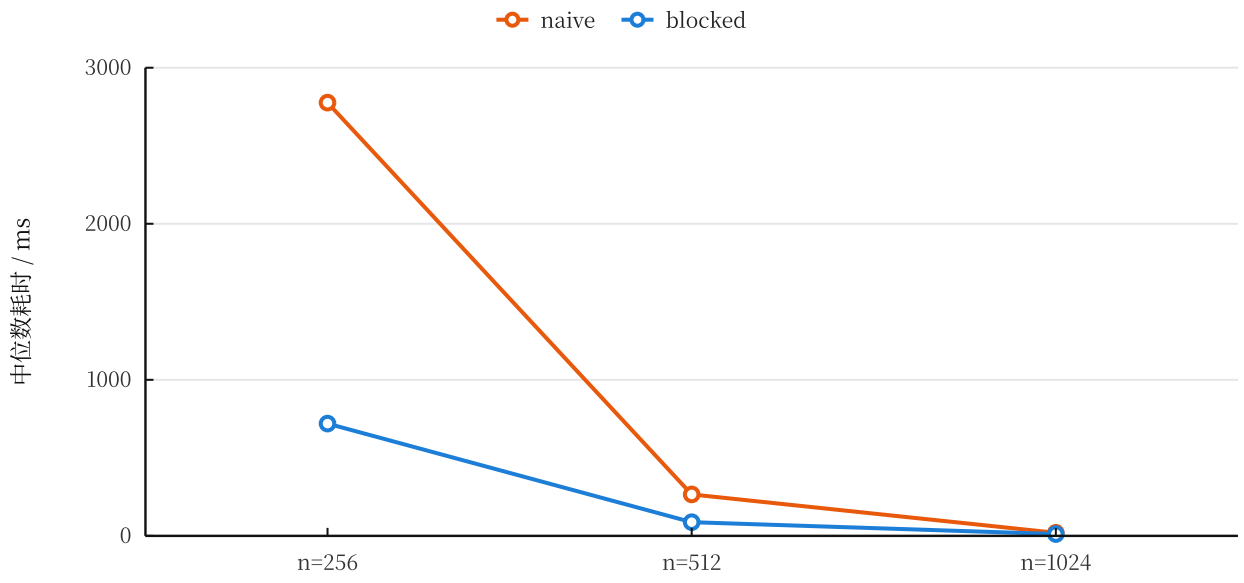


图 4 gemm 两种实现的中位数耗时随规模的变化（tile = 64）

（图中上方曲线为 `naive`，下方为 `blocked`。）

4.3 两种实现的规模伸缩性

N 翻倍时理论工作量增长 $2^3 = 8$ 倍。实测伸缩因子如下：

对比	理论因子	naive 实测	blocked 实测
256 → 512	8.00	265.565 ÷ 19.826 = **13.40**	87.712 ÷ 11.632 = **7.54**
512 → 1024	8.00	2776.290 ÷ 265.565 = **10.45**	719.567 ÷ 87.712 = **8.20**

`blocked` 的伸缩因子接近理论值 8（7.54 与 8.20），说明其每单位计算量的成本基本恒定，是一个"干净"的实现。

`naive` 的伸缩因子显著高于 8（13.40 与 10.45），即规模越大、单位计算量越吃亏。这正是 `naive` 的 GFLOP/s 随规模下降的直接体现，其成因见 4.5 节。

4.4 加速比为何随规模增长

第 2.4 节的流量模型给出了两种实现的搬运量：

N	naive 字节数（$8n^3$）	blocked 字节数（$12n^3/64 + 3n^2$）	模型预测流量比	实测加速比
256	134 MB	3.3 MB	**40.2×**	**1.70×**
512	1.07 GB	26.0 MB	**41.4×**	**3.03×**
1024	8.59 GB	204.5 MB	**42.0×**	**3.86×**

上述两列的对比图形化表示如下（柱为模型预测流量比，折线为实测加速比）：

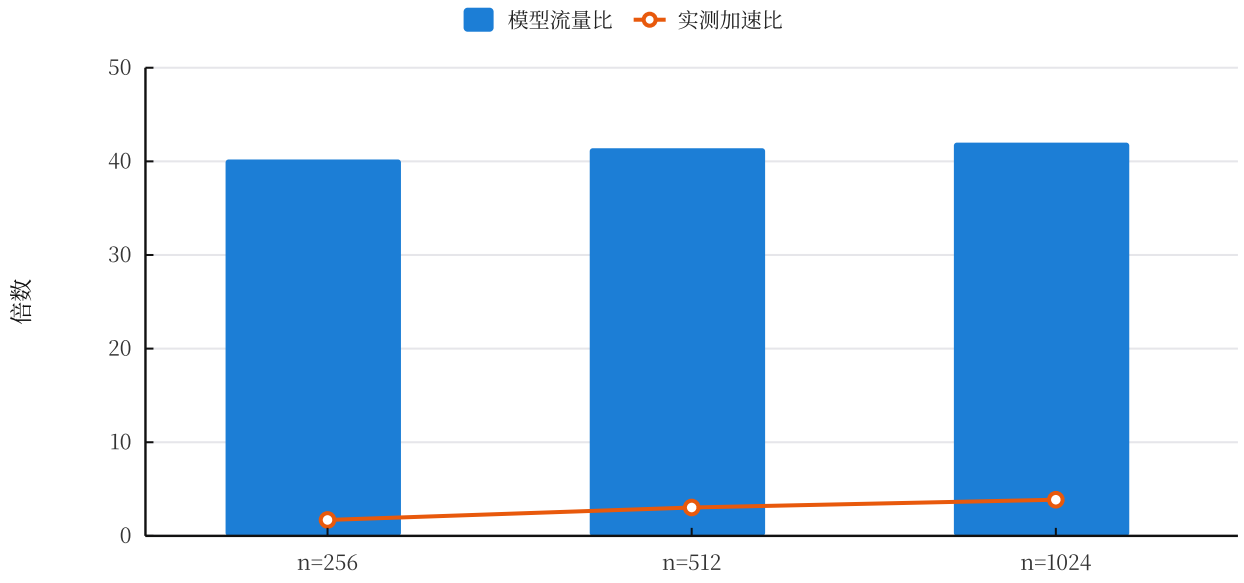


图 5 gemm 模型预测流量比与实测加速比（tile = 64）

图中柱高与折线之间的巨大落差即为本节的核心论点：模型预测两个实现应相差约 40 倍，实测却只有 1.70 ~ 3.86 倍。若两者都完全受限于内存带宽，柱与折线应当重合。

模型预测的流量比在三种规模下几乎不变（40 ~ 42×），而实测加速比却从 1.70× 涨到 3.86×。

这说明：**加速比并不由"流量比"决定**。若两种实现的耗时都完全由搬运量决定，加速比应当等于流量比（约 41×）；实测远低于此，且随规模变化。二者的差异只能来自**未被流量模型覆盖的成本项**，而该项在两种实现中的占比随规模变化。

`naive` 的伸缩因子高于理论值（4.3 节）即该成本项随规模上升的证据。最可能的来源是**缓存容量**：N=256 时三个矩阵约 0.75 MB 可部分驻留；N 增大后超出容量，跨行访问开始付出内存代价，而 `blocked` 的块内工作集（`tile=64` 时合计 48 KB）不受规模影响。

4.5 核心分析：瓶颈已不在内存

用修正模型还原有效带宽

按第 2 节的模型计算两种实现的实际搬运量，再与实测耗时相比：

N	实现	模型搬运量	实测耗时	由模型还原的带宽
256	`naive`	134 MB	19.826 ms	6.8 GB/s
256	`blocked`	3.3 MB	11.632 ms	0.29 GB/s
512	`naive`	1.07 GB	265.565 ms	4.0 GB/s
512	`blocked`	26.0 MB	87.712 ms	0.30 GB/s
1024	`naive`	8.59 GB	2776.290 ms	3.1 GB/s
1024	`blocked`	204.5 MB	719.567 ms	0.28 GB/s

`blocked` 的搬运算下来只相当于约 0.3 GB/s，而本机在 stencil 实验中实测到的可用带宽为 5.91 GB/s（第 3.4 节）。**`blocked` 只用了可用带宽的约 5%**。

> **结论：`blocked` 的耗时并非由内存带宽决定。** 分块把搬运量降到极低之后，时间仍然花在别处——即计算本身。

作为对照，`naive` 在 N=512 时还原出 4.0 GB/s，已接近可用带宽量级，说明它确实受制于访存；但其 GFLOP/s 随规模下降而非持平，表明访存与其它因素在共同作用。

瓶颈是标量指令吞吐

第 6 节将给出二进制层面的证据（反汇编中无任何 AVX 指令）。此处先看其后果：若内核完全标量，每个内层迭代只能完成 1 次乘加，则由主频直接给出吞吐上界：

```
标量 FMA 上限 ≈ 2 FLOP / 周期 × 主频
```

主频假设	标量上限	blocked 实测	达到上限的比例
1.6 GHz（基频）	3.2 GFLOP/s	2.98 ~ 3.06	**约 96%**
2.5 GHz	5.0 GFLOP/s	2.98 ~ 3.06	约 60%

`blocked` 的实测吞吐与**基频下的标量上限高度吻合**（约 96%）。这解释了两件事：

1. 为什么 `blocked` 的 GFLOP/s 在三种规模下都是约 3.0，几乎不变——它撞的是一堵**与规模无关的**墙，即标量指令吞吐。
2. 为什么 `naive` 的 GFLOP/s 更低（0.77 ~ 1.69）——它的内层访问（步长 `n`）无法连续取数，每个迭代的取数成本高于 `blocked`。

4.6 `tile` 扫描：调参空间已被算力上限封顶

固定 `N = 512`、`repeats = 5`，仅改变 `tile`：

tile	中位数	GFLOP/s	相对 tile = 8	模型流量	模型预测流量比
8	138.430 ms	1.94	1.00×	202.1 MB	5.3×
16	110.376 ms	2.43	1.25×	101.4 MB	10.6×
32	94.928 ms	2.83	1.46×	51.1 MB	21.0×
64	88.092 ms	3.05	1.57×	26.0 MB	41.4×
128	90.971 ms	2.95	1.52×	13.4 MB	80.3×
256	**86.539 ms**	**3.10**	**1.60×**	7.1 MB	151.7×

曲线形态：单调下降后趋于平台，而非 U 形

`tile` 从 8 增至 256，搬运量下降 **28 倍**（202 MB → 7.1 MB），但耗时只改善 **1.60 倍**。曲线在 `tile = 64` 之后基本平坦。

这与"存在最优 `tile`"的常见预期不同。 原因是 N=512 时整个问题的工作集始终未成为瓶颈：三个矩阵约 3 MB 可驻留 6 MB 的 L3，故"块太大导致缓存溢出"的效应在本实验规模内无法显现。

`tile` 的收益来自打包开销的摊销，且已接近极限

`gemm_blocked` 的计时包含打包（`at`、`bt` 的写入）与写回（`ct` 到 `c`）。打包工作量与输出块数成正比，即正比于 $1/\text{tile}^2$；而内层计算量为 n^3，与 `tile` 无关。因此总耗时可写成：

```
T(tile) ≈ a + b / tile
```

其中 `a` 为与 `tile` 无关的计算下限，`b/tile` 为打包与搬运的摊销项。用 `tile = 8` 与 `tile = 256` 两组实测值定参，得到：

参数	拟合值
`a`（计算下限）	**84.9 ms**

参数	拟合值
`b`（打包系数）	**428.5 ms · tile**

拟合值与全部六组实测的偏差都在 4% 以内：

tile	实测	拟合 a + b/tile	偏差
8	138.43 ms	138.43 ms	—（定参点）
16	110.38 ms	111.65 ms	−1.1%
32	94.93 ms	98.26 ms	−3.4%
64	88.09 ms	91.56 ms	−3.8%
128	90.97 ms	88.21 ms	+3.1%
256	86.54 ms	86.54 ms	—（定参点）

该拟合给出了两点量化结论：

1. **`tile = 8` 时，打包项占总耗时的 38.7%**（`428.5/8 ÷ 138.43`）；到 `tile = 256` 时降至 **1.9%**。这解释了曲线"先陡降"的部分。
2. **耗时下限 `a ≈ 84.9 ms` 与 `tile` 无关**，它就是 4.5 节所述的计算下限。曲线在 `tile ⩾ 128` 后趋平，正是因为打包项已不足 4%，**再增大 `tile` 也无法改善那 84.9 ms**。

结论

> `tile` 调参的全部收益上限约为 **1.6 倍**，且已被实测证实接近饱和。**继续优化 `tile` 的空间已经很小，因为瓶颈不在搬运量，而在与 `tile` 无关的计算下限（约 84.9 ms）**。 这与 4.5 节的判断一致：分块只解决了搬运问题，搬运问题解决后，剩下的全部是计算成本。

4.7 与 stencil 的横向对照

两个程序的两个"慢"实现有着同源的成因，但后果不同：

	stencil `column`	gemm `naive`
内层步长	`w` 个元素	`n` 个元素
缓存行利用	4/64 = 6.25%	同
相对同类实现的差距	**20.7×**（慢得多）	1.70 ~ 3.86×（慢得少）
原因	访存模式是**唯**一变量，惩罚充分体现	`naive` 同时受依赖链与访存限制，`blocked` 又受计算上限压制，两者差距被压缩

关键区别：stencil 的两种实现计算量、指令数完全相同（均已证实无 SIMD），差异纯粹来自访存，因此差距可以放大到 20 倍；而 gemm 的两种实现不仅访存不同、内层循环结构也不同（`naive` 为单一累加器链，`blocked` 内层有 `j` 方向的多个独立累加器），两者的瓶颈不同源，加速比因此被限制在 4 倍以内。

4.8 本节小结

1. **正确性**：三组规模的 `checksum` 在两种实现下逐位一致，尽管归约顺序不同。
2. **性能**：`blocked` 始终更快，加速比 **1.70× → 3.03× → 3.86×**，随规模单调增长。
3. **伸缩性**：`blocked` 的伸缩因子接近理论值 8（7.54 / 8.20）；`naive` 显著偏高（13.40 / 10.45），规模越大越吃亏。
4. **瓶颈转移**：模型流量比在三种规模下稳定在 40 ~ 42×，但实测加速比远低于此，说明加速比不由流量比决定。用修正模型还原后，`blocked` 的有效带宽仅约 **0.3 GB/s**（可用带宽的约 5%），**证明其瓶颈已**

不是内存带宽。

5. **瓶颈定性**：`blocked` 的实测吞吐（约 3.0 GFLOP/s）与**基频下的标量指令上限**吻合（约 96%），说明瓶颈是标量指令吞吐，且该上限与问题规模无关。二进制层面的证据见第 6 节。
6. **`tile` 调参**：曲线为"单调下降后趋平"而非 U 形；实测与 `T ≈ 84.9 + 428.5/tile` 吻合（偏差 < 4%）。`tile = 8` 时打包占 38.7%，`tile = 256` 时降至 1.9%。**调参收益上限约 1.6 倍且已趋饱和**，因为剩余耗时是与 `tile` 无关的计算下限。

---

5. 算术强度与 Roofline 分析

Roofline 模型用一个二维平面判断实现"本应"达到的性能：横轴是算术强度（每搬运一字节做了多少次浮点运算），纵轴是性能（GFLOP/s）。平面上有两条上界线——一条水平线代表机器的浮点吞吐上限，一条斜线代表带宽上限，两线交于"脊点"。

- 算术强度 **低于**脊点 → 实现受限于带宽，性能上限为 `强度 × 带宽`；
- 算术强度 **高于**脊点 → 实现受限于算力，性能上限为水平线。

5.1 平台上下限的确定

Roofline 需要两个平台参数。本报告不用理论规格值，而是用**本机上独立测得的两个实测值**作为上下限，理由是实测值已包含频率、微架构与内存子系统的综合影响。

参数	取值	依据
可用带宽上限 `BW`	**约 5.9 GB/s**	stencil `row` 实测：167 MB ÷ 28.298 ms = 5.91 GB/s。该实现的访问完全顺序、模型假设成立，可作为可持续带宽的估计
浮点吞吐上限 `P`	**约 4.4 GFLOP/s**	stencil `row` 实测 4.43 GFLOP/s。该实现与 GEMM 同为纯标量内核（第 6 节给出证据），可作为标量吞吐的估计

由此得到脊点：

```
脊点 = P ÷ BW = 4.43 GFLOP/s ÷ 5.91 GB/s ≈ 0.75 FLOP/Byte
```

关于这两个参数的两点说明：

1. **两者都取自同一个实测点**（stencil `row`）。这意味着该点必然同时"贴着"两条上界，因而**不能作为模型的独立验证**——它只是定义了模型的两个端点。模型的解释力应看其余实现在平面上的位置。
2. **`P ≈ 4.4 GFLOP/s` 是标量吞吐的估计值**。若内核被向量化，该上限应大幅提高（AVX2 理论上有 8 倍空间）。因此本节的 Roofline 描述的是**当前编译配置下的**平台，而非该 CPU 的硬件峰值。

5.2 各实现的算术强度

按第 2 节的公式计算（`tile = 64` 时 gemm blocked 取 $12n^3/tile + 3n^2$）：

实现	分子（FLOP）	分母（字节）	算术强度	相对脊点 0.75
stencil `row`（模型 A）	`6·updates`	`8·updates`	**0.75**	**恰在脊点**
stencil `column`（模型 B）	`6·updates`	`68·updates`	**0.088**	左侧，带宽受限
gemm `naive`	$2n^3$	$8n^3$	**0.25**	左侧，带宽受限
gemm `blocked`（`tile=8`，n=512）	$2n^3$	$1.506n^3$	**1.33**	右侧，算力受限
gemm `blocked`（`tile=64`，n=512）	$2n^3$	$0.193n^3$	**10.34**	右侧，算力受限
gemm `blocked`（`tile=256`，n=512）	$2n^3$	$0.0527n^3$	**37.93**	右侧，算力受限

算术强度把六个实现清晰地分成了两组：stencil `column` 与 gemm `naive` 在脊点左侧，受带宽约束；

gemm `blocked` 的全部 `tile` 取值都在脊点右侧，受算力约束。stencil `row` 恰好落在脊点上，属于过渡位置。

关于这些数值的两点说明：

1. **blocked 的算术强度依赖 `n`，不是仅由 `tile` 决定**。因为分母 $12n^3/tile + 3n^2$ 中的第二项在小 `n` 下不可忽略。表中数值按 `n = 512` 计算并已标注；同一 `tile` 下 `n` 越小，算术强度越低。第 2.5 节写"约等于 `tile/6`"是 `n/tile` 较大时的近似，此处采用完整公式。例如 `tile = 8` 时若忽略 $3n^2$ 项，会得到 5.33，而完整公式给出 1.33。
2. **即便 `tile = 8`，算术强度 1.33 也已高于脊点 0.75**，属于算力受限区间。这说明最小的分块也已把该实现推离带宽受限区——这与第 4.6 节"调参收益已饱和"互为印证：既然一开始就已在脊点右侧，继续增大 `tile` 只能改善搬运量，无法改善计算下限。

5.3 实测数据在平面上的位置

将各实现**实际达到**的带宽与算力同上下限比较：

实现	实际搬运量	实测耗时	实际带宽	带宽占上限	实测 GFLOP/s	算力占上限
stencil `row`	167 MB	28.298 ms	5.91 GB/s	**100%**	4.43	**100%**
stencil `column`	1420 MB	585.796 ms	2.42 GB/s	41%	0.21	4.7%
gemm `naive`（N=512）	1.07 GB	265.565 ms	4.04 GB/s	68%	1.01	23%
gemm `blocked`（N=512）	26.0 MB	87.712 ms	0.30 GB/s	**5%**	3.06	**69%**

四个实现呈现出四种截然不同的状态，这比单一结论更有信息量：

实现	状态	判读
stencil `row`	带宽与算力均接近上限	两个维度上都被充分利用（但见 5.4 节的限定）
stencil `column`	带宽仅 41%、算力仅 5%	**两个上限都没接近**——效率损失来自访存模式，而非某个硬件上限
gemm `naive`	带宽 68%、算力 23%	偏向带宽受限，但两者都不满
gemm `blocked`	带宽 5%、算力 **69%**	**明确受限于算力**——带宽完全不是约束

据此可对 stencil 的两个实现作出瓶颈定性：

- **`column` 受限于访存**，证据充分：算术强度 0.088 远在脊点左侧，实际搬运量放大约 8.5 倍，且带宽利用率仅 41%、算力利用率仅 4.7%——两个上限都未接近，效率损失来自访存模式本身。
- **`row` 位于脊点附近**（算术强度 0.75 ≈ 脊点 0.75），带宽与算力的利用率均为 100%，属于**计算与访存的过渡区**，倾向受限于计算。依据有二：其在三种规模下的 GFLOP/s 恒定（4.54 / 4.43 / 4.41），以及对 CPU 频率敏感（1.7 节）。

需要强调的是，`row` 的判定受 5.4 节边界三的限制：平台参数取自 `row` 自身，缺少独立标定，故上述定性为**倾向性结论**，而非确证。

5.4 三个需要显式声明的边界

Roofline 分析容易产生"一切都能被解释"的错觉。以下三点限制了本节结论的适用范围，必须声明。

边界一：`column` 未接近任何上限，说明模型漏掉了成本项

`stencil column` 的带宽占上限 41%，算力占上限仅 4.7%——**两个上限都远未接近**。若它真的只受带宽限制，应当达到约 5.9 GB/s；实测只有 2.42 GB/s。

这与第 3.4 节指出的残差是同一件事：**模型 B 预测总流量放大 8.5 倍，实测耗时放大 20.70 倍**。多出的部分无法用流量解释，推测来自访存延迟（`column` 的步长使预取器难以发挥作用）。此项未经硬件计数器验证。

边界二：gemm naive 的流量模型可能高估

`gemm naive` 的模型流量 $8n^3$ 假设 B 的每次访问都必然缺页。但在 N=256 与 N=512 时，矩阵总数据量较小，B 可能部分驻留缓存，实际 DRAM 流量低于模型值。因此表中其"实际带宽 4.04 GB/s"的置信度低于另外三行，**不宜作为精确结论引用**。

这一点也解释了第 4.3 节的现象：`naive` 的伸缩因子（13.40 / 10.45）高于理论值 8，正是因为规模较小时缓存提供了额外帮助，规模增大后该帮助消失。

边界三：模型参数只有两个点，无法独立验证

`BW` 与 `P` 均取自 stencil `row` 的实测，故 Roofline 的**形状**自洽，但**绝对位置**缺少独立标定（如用纯内存流式基准、纯计算循环分别测定）。本报告未包含该步骤。

因此本节结论限于"相对判读"，不宜断言"某实现达到硬件峰值的百分之多少"。

5.5 本节小结

1. **平台参数**：以本机实测取 `BW ≈ 5.9 GB/s`、`P ≈ 4.4 GFLOP/s`，脊点约 **0.75 FLOP/Byte**。二者取自同一实测点，故模型可自洽但未经独立标定。
2. **算术强度分组**：stencil `column`（0.088）与 gemm `naive`（0.25）位于脊点左侧，属带宽受限；gemm `blocked` 全部 `tile` 取值（`n=512` 时为 1.33 ~ 37.9）位于右侧，属算力受限；stencil `row`（0.75）恰在脊点。
3. **实测状态判读**：gemm `blocked` 带宽仅用 5%、算力用到 69%，**瓶颈明确的在算力一侧**；stencil `column` 两个上限都远未接近（41% / 4.7%），说明其效率损失来自访存模式而非硬件上限。
4. **三条边界**：`column` 的残差归因于延迟（推测）；`naive` 的流量模型在中小规模下可能高估；模型缺少独立标定实验。**本节结论限于相对判读**。

---

6. 编译器证据与优化决策

第 4.5 节从性能数据推断"瓶颈是标量指令吞吐"。这是一个**推断**，需要独立证据支持。本节给出三方面的证据，并据此给出优化决策。

6.1 SIMD 证据之一：编译器向量化报告

用 GCC 的向量化诊断选项重新编译（命令见附录 B），可得到每个循环是否被向量化的判定。与两个程序相关的关键条目如下。

`gemm.cpp`：

行号	循环	编译器的判定
12	`gemm_naive` 内层（`k`）	`not vectorized: not suitable for strided load` / `no vectype for stmt`
9、12	`gemm_naive` 外层	`not vectorized: multiple nested loops`
44	`gemm_blocked` 内层（`j`）	`not vectorized: no vectype for stmt: _34 = *_138; scalar_type: value_type`
42、36、34	`gemm_blocked` 其余层	`not vectorized: multiple nested loops` / `control flow in loop`

`stencil.cpp`：

行号	循环	编译器的判定
14	五点更新公式（`row` 分支）	`not vectorized: no vectype for stmt: _46 = *_45`
14	五点更新公式（`column` 分支）	`not vectorized: no vectype for stmt: _82 = *_81`
19、20	`column` 外层与内层	`control flow in loop` / `couldn't vectorize loop`
24、25	`row` 外层与内层	`control flow in loop` / `couldn't vectorize loop`

关键判定：

> **两个程序中，与热路径相关的循环没有任何一条产生 `optimized: ... vectorized` 记录。** 报告中出现的全部 `optimized` 条目均属于标准库（`stl_vector.h`、`stl_algobase.h` 等）与 `common.hpp` 中的统计代码，与 stencil 更新与矩阵乘法无关。

诊断输出本身**没有保存为文件**（当时直接输出到终端）。该输出可由附录 B 的命令随时重新生成，本节引用的判定与当时输出一致。

6.2 SIMD 证据之二：二进制中不存在 AVX 指令

编译器的报告是"意向声明"，二进制内容才是事实。用反汇编统计 256 位寄存器（`ymm`）的出现次数：

```
objdump -d stencil | grep -c 'ymm'   →   0
objdump -d gemm    | grep -c 'ymm'   →   0
```

两个可执行文件中均不存在任何 `ymm` 寄存器引用。 AVX/AVX2 的浮点向量运算必然使用 `ymm`（256 位），因此这一结果直接证明：**内核代码没有任何 SIMD 浮点指令**。该检查不依赖编译器报告，也不受机器状态影响。

6.3 SIMD 证据之三：关断 AVX2 的消融实验

若程序确实使用了 AVX2，关闭该指令集后性能应显著下降。用 `-mno-avx2` 重新编译（其余参数完全一致），并在**同一批次内**与原版交替运行：

二进制	中位数	标准差占比
`stencil`（`-march=native`）	34.306 ms	3.8%
`stencil-noavx`（`-mno-avx2`）	34.502 ms	7.3%

差异 0.6%，在测量噪声范围内。 关闭 AVX2 对性能没有可观测影响，与 6.2 节的结论一致。

关于该实验方法的一点说明：第一次尝试时，我比对的是"关断 AVX2 后的耗时"与"数十分钟前测得的原版耗时"，得到相差 27% 的结果。该对比**无效**，因为两次测量跨越了机器状态变化（见 1.7 节）。改为同批次交替运行后，差异消失。**这说明消融实验必须保证对照双方处于同一测量环境下，否则会把环境漂移误判为指令集差异。**

6.4 三条证据的汇合

证据	方法	结果
编译器判定	`-fopt-info-vec-optimized` / `-missed`	热路径循环无一被向量化
二进制内容	`objdump` 统计 `ymm`	计数为 **0**
行为消融	`-mno-avx2` 同批次对照	性能无变化（0.6%）

三者互相独立、结论一致：**在本报告的编译配置下，stencil 与 gemm 的全部四个实现均为纯标量内核。**

这为第 4.5 节的推断提供了依据：`blocked` 的实测吞吐（约 3.0 GFLOP/s）与基频下的标量上限（约 3.2 GFLOP/s）吻合，**因为内核确实是标量的。**

6.5 为什么编译器没能向量化

编译器的诊断信息给出了原因。分两类。

类 A：访问模式不合适（数据相关）

`gemm_naive` 内层的判定是 `not suitable for strided load`。内层语句为：

```
sum += a[i*n+k] * b[k*n+j];     // k 为内层变量
```

`b[k*n+j]` 在 `k` 方向前进时地址步长为 `n` 个元素。向量化该语句需要把 `n` 个不连续的元素装入一个向量，即"跨步加载"（gather）。**AVX2 没有通用的浮点 gather 指令**（`vgatherdps` 面向 32 位索引且吞吐很低，编译器通常判定不划算），因此该循环在结构上就无法向量化。

`stencil.cpp` 第 14 行的判定是 `no vectype for stmt`。该语句访问 `current[i-w]`、`current[i+w]`、`current[i-1]`、`current[i+1]` 四个不同偏移的地址，属于"多基址访问"，同样难以直接映射为向量加载。

类 B：可能存在可规避的障碍（推断，未验证）

除上述结构性限制外，诊断信息中还有若干条目提示存在**可能可以规避**的障碍。两点观察如下：其一，`gemm_blocked` 与 stencil 的判定中包含 `no vectype for stmt: ... scalar_type: const value_type`，对应的源码是被读取的标量声明为 `const`（如 `gemm.cpp` 第 43 行的 `const float value`）；其二，`stencil_step` 的两个指针参数（`stencil.cpp` 第 7 行）没有 `restrict` 限定，因此编译器必须保守假设"写入 `next[i]` 可能改变 `current` 的任意元素"，这与诊断中反复出现的 `control flow in loop` 相符。`gemm` 的接口使用 `std::vector` 引用，情况类似。

必须说明：以上两点为**基于诊断信息的推断，未经改写源码验证**（修改源码会破坏对照前提，见 1.6 节）。确认方法：对源码**副本**为 `stencil_step` 的指针参数加 `__restrict`，重编译后观察是否出现 `vectorized` 记录、`ymm` 计数是否非零。

6.6 优化决策

基于以上证据，可以给出**有依据的**优化方向，而非泛泛的建议。

决策一：stencil 的遍历顺序（收益最大、成本最低）

措施	依据	预期收益
将 `column` 改为 `row` 遍历	第 3.2 节：1024 与 2048 尺寸下稳定相差约 20.7 倍	**约 20 倍**

这是本报告发现的最大单项收益，无需改算法、无需向量化、无需额外内存，代价为零（两种遍历 checksum 逐位一致，见 3.1 节）。

决策二：GEMM 采用分块（收益随规模增长）

措施	依据	预期收益
用 `blocked` 替代 `naive`	第 4.2 节：随规模增长	N=256 时 1.70×；N=512 时 3.03×；N=1024 时 3.86×
`tile` 取值	第 4.6 节：`T ≈ 84.9 + 428.5/tile`	从 8 增至 256 约 1.6×，且 `tile ⩾ 128` 后趋平

决策要点：`tile` 的收益有上限且已接近饱和。**不应把优化预算继续投入 `tile` 调参**。

决策三：让内核向量化（唯一能突破当前上限的方向）

前三节的证据表明，当前所有实现都撞在**标量指令吞吐**这堵墙上（`blocked` 已达其 96%）。因此：

措施	说明	风险/成本
添加 `restrict` 限定	消除 6.5 节 B2 的别名障碍	需修改源码，破坏对照前提；须重做实验
显式多重累加器	`naive` 的单一 `sum` 形成串行依赖链；`blocked` 内层已有 `j` 方向的独立链，可借鉴	可能改变浮点结果（虽仍在容差内）
显式 SIMD 内建函数	用 AVX2 intrinsics 手写内层	工作量大，且**超出本任务范围**（本关为对照分析，不要求提交优化后的代码）

措施	说明	风险/成本
多线程	本机 `nproc` 为 2，理论收益上限约 2 倍	需引入并行库，且注意并行会改变归约顺序
Strassen 算法	降低运算复杂度至约 `O(n^2.807)`	数值稳定性下降，且实现复杂度高

关于"提高优化等级"这一常见建议：本报告**不建议**把它列为优化方向。原因有二。其一，诊断报告中的 `not vectorized: no vectype for stmt` 是在优化器**已经考虑过**该循环之后给出的判定，而非"未开启向量化"；其二，若更换优化等级，则必须对两种实现同时更换并重做全部实验，否则违反 1.2 节的参数一致性要求。因此该措施即便有效，也不属于本报告证据能支持的结论。

必须强调：以上措施均**未在本报告中实施**。原因见 1.6 节——一旦修改源码，"`naive` 与 `blocked`"的对照前提即被破坏，本报告的全部实测数据将无法解释。这些方向属于**基于现有证据的后续工作**，而非本报告的结论。

6.7 本节小结

1. **三方面证据汇合**：编译器报告（无 `vectorized` 记录）、反汇编（`ymm` 计数为 0）、消融实验（关断 AVX2 无影响），共同证明**四个实现全部为纯标量内核**。这为第 4.5 节的推断提供了独立依据。
2. **未向量化的原因分两类**：数据相关（`naive` 的跨步访问、stencil 的多基址访问，属结构限制）与实现相关（类型转换、缺少 `restrict`，属可规避障碍）。第二类给出了可验证的最小实验方案。
3. **优化决策三条**：改遍历顺序（约 20 倍，成本为零）；采用分块（1.70 ~ 3.86 倍，随规模增长）；让内核向量化（唯一能突破标量上限的方向）。`tile` 调参已被证实接近饱和，不应继续投入。
4. **本次实验未实施任何优化**，以保全对照实验的有效性；上述方向列为后续工作。

---

7. 结论与局限性

7.1 主要结论

结论一：两个"慢实现"的成因同源，但后果相差悬殊

stencil 的 `column` 与 gemm 的 `naive` 具有同一个病根：**内层循环的地址步长为一个整行宽度**（分别为 `w` 与 `n`），导致每次读取只使用缓存行的 4/64 = 6.25%，读流量放大约 16 倍。

但两者的性能后果完全不同：

	stencil `column`	gemm `naive`
与同类实现的差距	**慢 20.70 倍**（1024 尺寸）	慢 3.86 倍（N=1024）
差异来源	**仅访存顺序变化**，计算量、指令数完全相同	访存顺序与内层循环结构**同时变化**

stencil 的对照实验是"单一变量"的：两种遍历的 checksum 逐位一致，且均已证实为纯标量内核，因此 20.70 倍的差距可以完整归因于访存模式。**这是本报告中最干净的因果证据**。

结论二：`column` 慢的原因是"搬得多"，而不是"带宽不够"

程序输出的 `effective_bandwidth_GB/s` 对 `column` 给出的数值（0.285 GB/s）**低于硬件的物理下限**（该值比机械硬盘还慢），不能作为"带宽不足"的证据。其失效原因是公式分子 `8 × updates` 是基于"理想空间复用"的常量，与遍历顺序无关。

用修正模型还原后：

实现	实际搬运量	实测耗时	真实带宽	程序报出的"有效带宽"
`row`	167 MB	28.298 ms	**5.91 GB/s**	5.91 GB/s
`column`	1420 MB	585.796 ms	**2.42 GB/s**	0.285 GB/s ❌

两者真实带宽处于同一量级（相差 2.4 倍），而程序报出的数值相差 20.7 倍。 差异的实质是搬运量的放大，而非带宽利用效率的差异。

结论三：GEMM 的瓶颈已从内存转移到标量算力

分块把搬运量降低约 42 倍（N=512、tile=64 时 1.07 GB → 26 MB），但实测加速比只有 3.03 倍。换算后发现：

- `blocked` 的实际带宽仅约 **0.30 GB/s**，为可用带宽（5.91 GB/s）的 **5%**；
- 其 GFLOP/s 为 3.06，达到**基频下纯标量指令上限**（约 3.2 GFLOP/s）的 **96%**。

分块解决了搬运问题，之后剩下的全部是计算成本，而计算被标量指令吞吐封顶。 这解释了一个看似矛盾的现象：`blocked` 在三种规模下的 GFLOP/s 几乎不变（2.88 / 3.06 / 2.98）——它撞的是一堵与规模无关的墙。

结论四：stencil 两种实现的瓶颈分属不同维度

与结论三（gemm）对应：stencil 的 `column` 受限于**访存**，`row` 则倾向受限于**计算**。两者的共同点是——**一旦访存瓶颈被消除（顺序化或分块），剩余成本全部落在计算上**。这一结构在 stencil 与 gemm 两个程序中一致出现，是本报告最主要的横向观察。

详见第 5.3 节的定性依据与 5.4 节的适用边界。

结论五：tile 调参的空间已经饱和

实测与 `T(tile) ≈ 84.9 + 428.5/tile`（毫秒）吻合，六组数据偏差均在 4% 以内。其中：

- `428.5/tile` 为打包与搬运开销：`tile = 8` 时占总耗时 **38.7%**，`tile = 256` 时降至 **1.9%**；
- **`84.9 ms` 是与 `tile` 无关的计算下限**。

因此 `tile` 从 8 增至 256 的全部收益约为 **1.6 倍且已趋平。继续投入 `tile` 调参不会带来实质改善**，因为剩余耗时由计算下限决定。

结论六：当前配置下全部实现均为纯标量，这是性能的共同天花板

三方面独立证据汇合：

证据	结果
编译器向量化诊断	热路径循环**无**一产生 `vectorized` 记录
二进制反汇编	`ymm` 寄存器计数为 **0**
`-mno-avx2` 同批次消融	性能**无变化**（差异 0.6%，在噪声内）

这也解释了为何两个实现组的 GFLOP/s 都远低于 CPU 的向量峰值。 由于该结论有硬证据支撑，它同时限定了前四条结论的适用范围：它们描述的是**当前编译配置下**的性能结构。

7.2 交付指标对照

要求指标	结果	位置
正确性结果	stencil 三组尺寸、gemm 三组规模，成对实现的 `checksum` **逐位一致**；全部 `check=PASS`；`max_abs_error` 在 10^{-7}（FP32 stencil）与 10^{-4}（N=1024 gemm）量级	3.1、4.1 节
平均时间	见附录 A 的 `平均 (ms)` 列	附录 A
中位数	见附录 A 的 `中位数 (ms)` 列；所有吞吐指标均以中位数为基准（`throughput_time=median`）	3.2、4.2、附录 A
总体标准差	见附录 A 的 `总体标准差` 与 `标准差占比` 两列；程序使用总体标准差（除以 `n` 而非 `n-1`）	附录 A

要求指标	结果	位置
GFLOP/s	stencil：`row` 4.41 ~ 4.54，`column` 0.21 ~ 1.11；gemm：`naive` 0.77 ~ 1.69，`blocked` 2.88 ~ 3.06	3.2、4.2、附录 A
有效带宽	stencil 程序输出该指标，但**对 `column` 失效**（3.4 节给出辨析与修正）；gemm 程序不输出该指标，由本报告按流量模型还原	3.4、4.5、附录 A

关于有效带宽：stencil 程序中该指标定义为 `8 × updates ÷ median_ms`，分子基于理想空间复用假设；3.4 节已论证该假设对 `column` 不成立。gemm 未提供该指标，由第 2.3 节模型计算。**上述数值均为模型推导值，非硬件计数器实测值**。

7.3 局限性

1. **未使用硬件计数器**，故以下两点为推测：`column` 在模型预测（8.5 倍）与实测（20.70 倍）间约 2.4 倍残差归因于访存延迟；`row` 与 `column` 稳定性差异（跨度 21% 与 12%）归因于对 CPU 频率的敏感度不同。
2. **Roofline 缺少独立标定**：平台参数 `BW` 与 `P` 均取自 stencil `row` 这一个实测点，该点必然贴合两条上界，故不能用于验证模型。第 5 节结论限于**相对判读**。此外 `gemm naive` 的流量模型在中小规模下可能高估（$8n^3$ 假设每次访问都缺页）。
3. **未验证编译器未向量化的具体障碍**，6.5 节类 B 的两点为基于诊断信息的推断。**未实施任何代码优化**，6.6 节的优化决策为方向性判断，其收益为推算值。
4. **测量存在环境漂移**：同一二进制的 `stencil row` 中位数在不同批次间相差约 21%（推测为热降频）。处理方式见 1.7 节——主要结论基于同批次配对数据，不跨批次比较绝对耗时。

7.4 后续工作

按预期收益排序：① **改遍历顺序**（`column` → `row`，成本近乎为零，预期约 20 倍）；② **采用分块并选定 `tile`**（1.70 ~ 3.86 倍，随规模增长，`tile` 取 64 ~ 256 即可）；③ **使内核向量化**（唯一能突破标量上限的方向，建议先验证 6.5 节类 B 的障碍）；④ **补充独立标定实验**（纯内存带宽基准与纯计算基准），并可尝试用硬件计数器验证访存延迟假设。

附录 A：原始数据表

以下数据由 `results/` 目录中的 22 个原始输出文件提取，未做任何修改。所有运行均满足 `check=PASS`。

A.1 实验数据（共 22 个原始文件，均满足 check=PASS）

配置	中位数 (ms)	标准差占比	备注
stencil `row` @ 256^2	1.7041	9.17%	
stencil `column` @ 256^2	6.9732	3.35%	
stencil `row` @ 1024^2	28.2976	0.30%	主结论配对
stencil `column` @ 1024^2	585.7960	0.54%	主结论配对
stencil `row` @ 2048^2	114.0010	1.68%	
stencil `column` @ 2048^2	2354.1900	0.87%	
gemm `naive` n=256	19.8256	2.17%	GFLOP/s 1.69
gemm `blocked` n=256, tile=64	11.6322	6.87%	2.88
gemm `naive` n=512	265.5650	0.83%	1.01
gemm `blocked` n=512, tile=64	87.7116	0.17%	3.06

配置	中位数 (ms)	标准差占比	备注
gemm `naive` n=1024	2776.2900	1.00%	0.77
gemm `blocked` n=1024, tile=64	719.5670	0.05%	2.98
gemm `blocked` n=512, tile=8/16/32	138.43 / 110.38 / 94.93	⩽0.48%	GFLOP/s 1.94/2.43/2.83
gemm `blocked` n=512, tile=128/256	90.97 / 86.54	⩽1.09%	2.95 / 3.10

另有三组 1024^2 重测数据用于稳定性验证：`row` 28.7095 ms（0.37%）、`column` 621.09 ms（10.95%）、`column` 630.08 ms（3.21%）；以及同批次配对的 `row` 34.8139 ms（3.37%）。各组 checksum 与其对应尺寸一致。

A.2 成对汇总

stencil（steps=20）：256^2 为 1.7041 → 6.9732 ms（**4.09**×）；1024^2 为 28.2976 → 585.7960 ms（**20.70**×）；2048^2 为 114.0010 → 2354.1900 ms（**20.65**×）。

gemm（tile=64）：n=256 为 19.8256 → 11.6322 ms（**1.70**×）；n=512 为 265.5650 → 87.7116 ms（**3.03**×）；n=1024 为 2776.2900 → 719.5670 ms（**3.86**×）。

A.3 `tile` 扫描汇总（N = 512，`blocked`）

`tile` 取 8 / 16 / 32 / 64 / 128 / 256 时，中位数分别为 138.43 / 110.38 / 94.93 / 88.09 / 90.97 / 86.54 ms，相对 `tile = 8` 的加速比为 1.00× / 1.25× / 1.46× / 1.57× / 1.52× / 1.60×；对应模型流量为 202.1 / 101.4 / 51.1 / 26.0 / 13.4 / 7.1 MB。拟合式 `T ≈ 84.9 + 428.5/tile`，六点偏差均小于 4%。详见 4.6 节。

A.4 数据溯源说明

`A1-1024-row-column-same-batch.txt` 由同一次连续运行的两条命令输出合并而成（`row` 与 `column` 各一次），其内容与终端输出一致。该文件未在测量当时通过 `tee` 直接落盘，而是在整理报告阶段依据原始终端输出补建。其余 21 个文件均为测量当时由 `tee` 直接写入 `results/` 目录。

附录 B：复现命令

B.1 编译与实验命令

```
# 环境记录
g++ --version | head -1 ; nproc ; grep -m1 'model name' /proc/cpuinfo

# 编译（两程序参数必须一致）
g++ -std=c++17 -O2 -march=native -o stencil stencil.cpp
g++ -std=c++17 -O2 -march=native -o gemm    gemm.cpp

# 用法
./stencil HEIGHT WIDTH STEPS row|column REPEATS
./gemm    N naive|blocked TILE REPEATS

# 实验（共 22 组，示例）
mkdir -p results
./stencil 1024 1024 20 row    5 | tee results/A1-1024-row.txt
./stencil 1024 1024 20 column 5 | tee results/A1-1024-column.txt
./gemm    512  naive   64 5 | tee results/B1-512-naive.txt
./gemm    512  blocked 64 5 | tee results/B1-512-blocked.txt
./gemm    512  blocked 32 5 | tee results/B2-tile-32.txt
# 其余组合同理：stencil 改尺寸与 row|column；gemm 改 n、impl 与 tile
```

B.2 第 6 节三份证据的采集命令

```
# 证据一：编译器向量化诊断
g++ -std=c++17 -O2 -march=native -fopt-info-vec-optimized -fopt-info-vec-missed \
    -o /dev/null gemm.cpp 2>&1 | grep 'gemm.cpp'

# 证据二：反汇编统计 AVX 寄存器（预期输出 0）
objdump -d stencil | grep -c 'ymm'
objdump -d gemm    | grep -c 'ymm'

# 证据三：消融实验（必须同批次交替运行，否则会把环境漂移误判为指令集差异）
g++ -std=c++17 -O2 -march=native -mno-avx2 -o stencil-noavx stencil.cpp
./stencil       1024 1024 20 row 5
./stencil-noavx 1024 1024 20 row 5
```

B.3 工具可用性检查（可选）

```
perf --version
perf list       # 确认本机支持哪些事件
```

注意：`perf stat` 为整进程统计，包含 FP64 参考计算、校验与打印开销，**不可将其结果除以本报告中的 `compute_ms`**。若要获得内核区域的精确计数，需另行实现区域采集（当前示例程序未实现）。

项目地址

本报告及全部实验代码、原始测量数据已开源于：

https://github.com/2jo7qOii/hpc-stencil-gemm

仓库包含：被测程序源码（`stencil.cpp`、`gemm.cpp`、`common.hpp`）、22 个原始测量输出（`results/`）、报告源文件与打印样式（`report/`）。